

Relatório de Auditoria de Segurança

Collegium (Solo Learning) — Re-auditoria (v2)

Data: 28 de agosto de 2026

Escopo: Backend (Fastify + Prisma + PostgreSQL) — Verificação de correções aplicadas

15

Total Original

8

Corrigidos ■

7

Abertos ■■

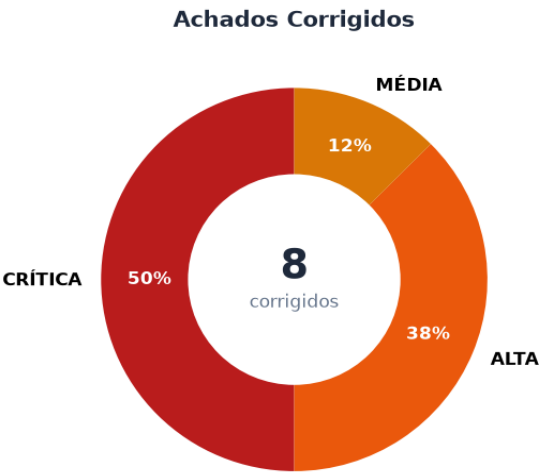
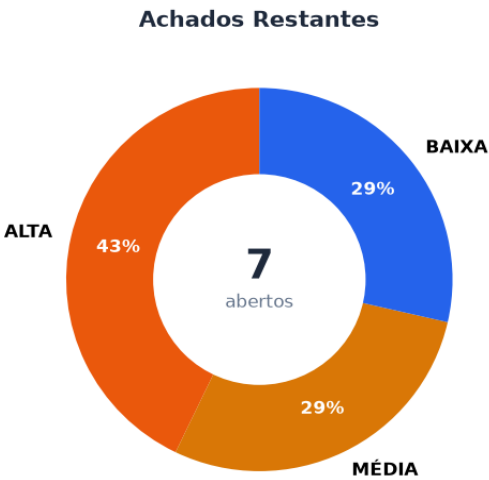
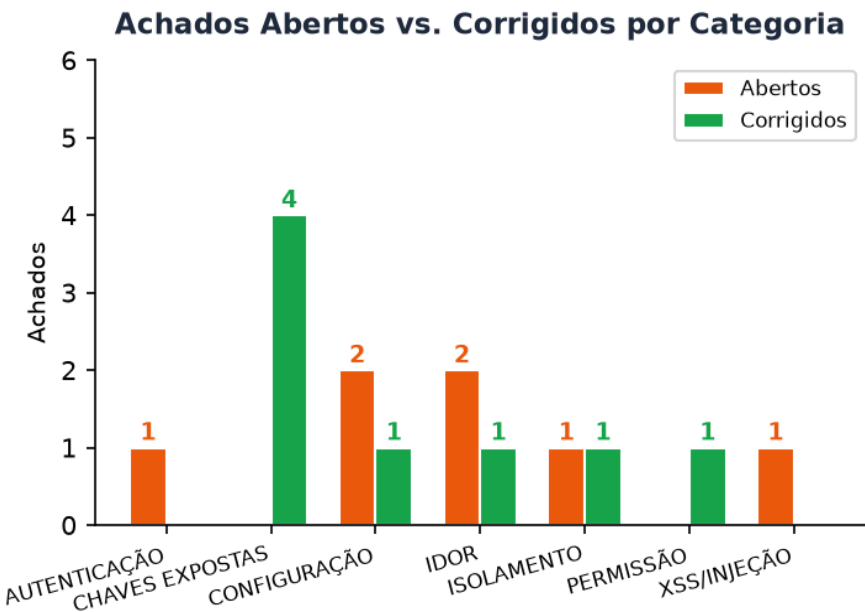
Taxa de correção: 53% dos achados originais foram resolvidos.

1. Resumo Executivo

Esta re-auditoria verifica as correções aplicadas aos 15 achados da auditoria original (v1). Dos 15 achados, **8 (53%) foram corrigidos e 7 permanecem abertos**.

Achados Restantes por Severidade:

Severidade	Abertos	Corrigidos
■ Crítica	0	4
■ Alta	3	3
■ Média	2	1
■ Baixa	2	0



2. Correções Aplicadas (■ Corrigidos)

Dos 15 achados originais, 8 foram corrigidos. Detalhes abaixo:

■ CORRIGIDO CRED-001 ■ CRÍTICA

Segredo JWT hardcoded como fallback → REMOVIDO

■ backend/src/plugins/auth.ts:6-8

ANTES: fallback hardcoded 'supersecret_solen_key_123'. AGORA: validação no startup que aborta se JWT_SECRET não estiver definido.

```
// ANTES: secret: process.env.JWT_SECRET || 'supersecret_solen_key_123' // DEPOIS: const jwtSecret = process.env.JWT_SECRET; if (!jwtSecret) { throw new Error('FATAL: JWT_SECRET not defined'); } fastify.register(fastifyJwt, { secret: jwtSecret });
```

■ CORRIGIDO CRED-003 ■ CRÍTICA

Credenciais PostgreSQL hardcoded → Agora usa variáveis de ambiente

■ backend/docker-compose.yml:5-7

ANTES: valores inline. AGORA: usa \${POSTGRES_USER:-default}. Melhor, mas defaults ainda fracos.

```
# ANTES: POSTGRES_USER: solen_user # DEPOIS: POSTGRES_USER: ${POSTGRES_USER:-solen_user}
```

■ CORRIGIDO CRED-004 ■ CRÍTICA

Senhas de alunos em texto plano → Agora hasheadas com bcrypt

■ backend/src/routes/admin.ts:452,504,549

Todos os 3 pontos (create, batch, reset) agora usam bcrypt.hash() antes de persistir.

```
// ANTES: password: 'INITIAL_SUMMONING_CODE_LOGIN' // DEPOIS: const initialPasswordHash = await bcrypt.hash('SUMMONING_CODE', 10); password: initialPasswordHash
```

■ CORRIGIDO CRED-005 ■ ALTA

Senha padrão hardcoded 'Solen2026' → Agora gera senha aleatória

■ backend/src/routes/superadmin.ts:60,163

ANTES: fallback 'Solen2026'. AGORA: gera senha aleatória via crypto.randomBytes e retorna temporaryPassword ao admin.

```
// ANTES: const rawPassword = password || 'Solen2026'; // DEPOIS: const rawPassword = password || `Solen#${crypto.randomBytes(4).toString('hex')}`; // Retorna temporaryPassword no response
```

■ CORRIGIDO AUTH-001 ■ CRÍTICA

Webhook SEM autenticação → Agora valida WEBHOOK_SECRET

■ backend/src/routes/webhooks.ts:1-20

ANTES: zero autenticação. AGORA: valida header x-webhook-secret (ou asaas-access-token, stripe-signature) contra WEBHOOK_SECRET.

```
const webhookSecret = process.env.WEBHOOK_SECRET; const incomingSecret = request.headers['x-webhook-secret']; if (webhookSecret && incomingSecret !== webhookSecret) { return reply.status(401).send({ error: 'Não autorizado' }); }
```

■ CORRIGIDO IDOR-003 ■ ALTA**Matriz de auditoria cross-tenant → Agora restringe a ADMIN****■ backend/src/routes/admin.ts:matrix/audit**

ANTES: institutionId/instituicao aceitos de qualquer usuário. AGORA: só ADMIN pode usar esses parâmetros.

```
const isSuperAdmin = request.user.role === 'ADMIN'; if (isSuperAdmin && institutionId) { targetInst = await prisma.institution.findUnique(...); } else if (request.user.institutionId) { targetInst = await prisma.institution.findUnique(...); }
```

■ CORRIGIDO ISOLATION-001 ■ ALTA**GET /bounty/active sem filtro → Agora filtra por instituição****■ backend/src/routes/bounty.ts:123-135**

ANTES: retorna bugs de todas instituições. AGORA: filtra por instituicao para não-ADMIN.

```
where: request.user.role === 'ADMIN' ? {} : (userInst ? { instituicao: userInst } : {})
```

■ CORRIGIDO CORS-001 ■ MÉDIA**CORS origin '*' → Agora usa CORS_ORIGIN env var****■ backend/src/server.ts:14-17**

ANTES: hardcoded '*'. AGORA: lê CORS_ORIGIN de variável de ambiente, fallback para '*' se não definido.

```
const allowedOrigins = process.env.CORS_ORIGIN ? process.env.CORS_ORIGIN.split(',').map(o => o.trim()) : '*'; server.register(cors, { origin: allowedOrigins });
```

3. Achados Restantes (■■ Abertos)

Restem 7 achados não corrigidos:

■ IDOR

IDOR-001 ■ ALTA

Reatribuição de aluno a outra instituição sem verificar posse

■ backend/src/routes/admin.ts:441-460

Ao cadastrar um aluno via POST /admin/stomes, se a matrícula já existe, o código atualiza turmaId, turno, instituicao e institutionId SEM verificar se o aluno pertence à mesma instituição do Arquiteto.

```
if (existingStudent) { const updatedStudent = await prisma.user.update({ where: { id: existingStudent.id }, data: { turmaId, turno, instituicao: request.user.instituicao, institutionId: request.user.institutionId || null } }); // ■■ Sem verificação de posse!
```

IDOR-002 ■ ALTA

Cadastro em lote não verifica instituição de matrículas existentes

■ backend/src/routes/admin.ts:560-592

O cadastro em lote POST /admin/stomes/batch não verifica se alunos com matrículas existentes pertencem à instituição do Arquiteto.

```
for (const s of students) { await prisma.user.create({ data: { matricula: s.matricula.toLowerCase().trim(), // ■■ Sem verificação de instituição para matrículas existentes } }); }
```

■ ISOLAMENTO

ISOLATION-002 ■ MÉDIA

Alunos não atribuídos listados globalmente para Professores

■ backend/src/routes/professor.ts:69-80

Quando um Professor consulta GET /professor/stomes?unassigned=true, a query retorna TODOS os alunos sem turma, sem filtro de instituição.

```
if (unassigned === 'true') { where.turmaId = null; // ■■ Sem filtro de turmaDisciplinas ou instituição }
```

■ XSS/INJEÇÃO

XSS-001 ■ ALTA

Input do usuário injetado em HTML de e-mail sem sanitização

■ backend/src/routes/bounty.ts:55

A descrição do bug report é injetada diretamente no template HTML do e-mail com apenas replace de \n por , sem escape de HTML.

```
html: ` ${description.replace(/\n/g, ' ')} ` // ■■ Sem escape HTML
```

■ AUTENTICAÇÃO

AUTH-002 ■ MÉDIA

Token JWT aceito via query string (?token=...)

■ backend/src/plugins/auth.ts:12-15

O plugin de autenticação aceita tokens na query string além do header Authorization. Tokens em URLs ficam em logs e histórico.

```
const queryToken = (request.query as any)?.token; if (queryToken && !request.headers.authorization) {  
  request.headers.authorization = `Bearer ${queryToken}`; }
```

■ CONFIGURAÇÃO

CONFIG-001 ■ BAIXA

Senha padrão '1234' para professores (fraca)

■ backend/src/routes/admin.ts:84

A senha padrão para novos professores continua sendo '1234'. Embora agora hasheada com bcrypt, é trivialmente adivinhável na janela antes do primeiro acesso.

```
const defaultPassword = await bcrypt.hash('1234', 10);
```

CONFIG-002 ■ BAIXA

Código de invocação padrão '1234' para turmas

■ backend/src/routes/admin.ts:134

O código de invocação padrão para novas turmas continua sendo '1234'.

```
codigoInvocacao: codigoInvocacao ? codigoInvocacao.trim() : "1234"
```

4. Recomendações Priorizadas

P1 — URGENTE

- IDOR-001: Adicionar verificação ``existingStudent.instituicao === request.user.instituicao`` antes de reatribuir aluno.
- XSS-001: Escapar HTML na descrição do bug report antes de injetar no template de e-mail.

P2 — ALTO

- IDOR-002: Verificar se matrículas existentes no batch pertencem à mesma instituição.
- ISOLATION-002: Filtrar alunos não atribuídos por instituição do Professor.
- AUTH-002: Remover suporte a token via query string em produção.

P3 — MÉDIO

- CONFIG-001: Gerar senhas aleatórias para professores (como já feito para arquitetos).
- CONFIG-002: Forçar alteração de código de invocação na primeira criação de turma.

5. Issues para o GitHub

Issues para os achados restantes (os já corrigidos não precisam de issue):

```
--- ISSUE 1 ---

### [Segurança] IDOR: reatribuição de alunos entre instituições via POST /admin/stomes

**Labels:** security, high

## Descrição
No endpoint POST /admin/stomes, quando a matrícula já existe, o código atualiza o aluno SEM verificar se pertence à mesma instituição do Arquiteto.

## Evidência
```typescript
// backend/src/routes/admin.ts:441-460
if (existingStudent) {
 const updatedStudent = await prisma.user.update({
 where: { id: existingStudent.id },
 data: {
 turmaId, turno,
 instituicao: request.user.instituicao, // ■■ Sem checar posse!
 institutionId: request.user.institutionId || null
 }
 });
}
}
```

## Impacto
Um Arquiteto da instituição A pode assumir alunos da instituição B.

## Sugestão
Adicionar: `if (existingStudent.instituicao !== request.user.instituicao) return reply.status(403)...`

## Critérios de Aceite
- [ ] Verificação de posse antes de reatribuir
- [ ] Teste com cross-tenant IDOR

--- FIM ISSUE 1 ---
```

```
--- ISSUE 2 ---

### [Segurança] IDOR: cadastro em lote não verifica instituição de matrículas existentes

**Labels:** security, high

## Descrição
POST /admin/stomes/batch não verifica se matrículas existentes pertencem à mesma instituição.

## Evidência
```typescript
// backend/src/routes/admin.ts:560+
```

```
for (const s of students) {
 await prisma.user.create({
 data: { matricula: s.matricula.toLowerCase().trim(), ... }
 // ■■ Sem verificação cross-tenant
 });
}
...

Impacto
Mesmo impacto do IDOR-001, mas em escala (lote).

Critérios de Aceite
- [] Verificar instituição de matrículas existentes antes de cadastrar

--- FIM ISSUE 2 ---
```

```
--- ISSUE 3 ---

[Segurança] Isolamento: alunos não atribuídos listados globalmente para Professores

Labels: security, medium

Descrição
GET /professor/stomes?unassigned=true retorna todos os alunos sem turma de todas as instituições.

Evidência
```typescript
if (unassigned === 'true') {
  where.turmaId = null;
  // ■■ Sem filtro de turmaDisciplinas ou instituição
}
...

## Critérios de Aceite
- [ ] Filtrar por turma: { instituicao } para PROFESSOR quando unassigned=true

--- FIM ISSUE 3 ---
```

```
--- ISSUE 4 ---

### [Segurança] XSS via e-mail: input injetado em HTML sem sanitização

**Labels:** security, high

## Descrição
A descrição do bug é injetada no HTML do e-mail sem escape.

## Evidência
```typescript
html: `
<blockquote>
${description.replace(/\n/g, '
')}
</blockquote>
`
```

```
...
```

```
Critérios de Aceite
```

```
- [] Input sanitizado (escape de <, >, & antes de inserir no HTML)
```

```
--- FIM ISSUE 4 ---
```